

DETECTION ENGINEERING ANALYSIS

Lazarus Group · Ronin Network Hack · MITRE ATT&CK Framework

1. INTRODUCTION

DETECTION ENGINEERING ANALYSIS OF LAZARUS GROUP (RONIN HACK) USING MITRE ATT&CK

1. INTRODUCTION

The Lazarus Group is a North Korea-linked threat actor known for high-profile cyberattacks, especially against financial institutions and cryptocurrency platforms.

In March 2022, Lazarus Group executed the Ronin Network Hack, stealing over \$600 million in cryptocurrency from the Ronin Bridge — the blockchain platform used by the game Axie Infinity.



The **Lazarus Group** is a North Korea-linked threat actor behind some of the largest financial cyberattacks in history. In March 2022, they executed the **Ronin Network Hack**, stealing over **\$600 million** in cryptocurrency from the Ronin Bridge — the blockchain platform used by the game Axie Infinity.

Weaknesses in access control and monitoring allowed attackers to seize control of validator nodes and authorise fraudulent transfers without triggering a single alert. This report maps the attack to the **MITRE ATT&CK framework** and delivers detection logic for SOC teams.

2. ATTACK OVERVIEW

This project analyzes the attack using the MITRE ATT&CK framework and identifies detection opportunities for defenders.

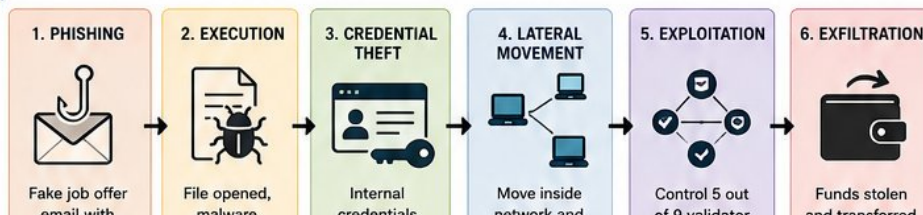
LAZARUS GROUP

RONIN NETWORK

STOLEN: \$600M+

2. ATTACK OVERVIEW (RONIN HACK)

1. **Phishing:** Employee receives fake job offer with malicious file.
2. **Execution:** File is opened, malware is installed.
3. **Credential Theft:** Attackers steal internal credentials.
4. **Lateral Movement:** Access to validator nodes and internal systems.
5. **Exploitation:** Control of 5 out of 9



- **Target:** Sky Mavis (developer of Axie Infinity)
- **Method:** Social engineering + credential compromise
- **Impact:** ~\$600M stolen in cryptocurrency

1. Phishing	Fake job offer email with malicious PDF sent to a Sky Mavis employee.
2. Execution	Employee opens the file; malware runs silently in the background.
3. Credential Theft	Internal credentials stolen, targeting validator node accounts.
4. Lateral Movement	Attackers move across the network, gaining control of more nodes.
5. Exploitation	5 of 9 validators compromised — enough to approve any transaction.
6. Exfiltration	Cryptocurrency transferred out; no alerts triggered throughout.

3. MITRE ATT&CK; MAPPING

validators gained.
6. **Exfiltration:** Cryptocurrency funds are stolen and transferred out.

malicious file

executed

stolen

access systems

nodes

out

3. MITRE ATT&CK MAPPING

Mapping the attacker's behavior to MITRE ATT&CK techniques helps us understand the tactics used and where detection is possible.

PHASE	WHAT HAPPENED	MITRE ATT&CK TECHNIQUE	TECHNIQUE ID	WHAT IT MEANS
 Initial Access	Phishing email	Phishing Attachment	T1566.001	Attacker tricks user to open file
 Execution	File opened	User Execution	T1204	Malware runs when user opens file
 Execution	PowerShell used	PowerShell	T1059.001	Hidden scripts run via PowerShell
 Persistence	Scheduled task created	Scheduled Task	T1053.005	Attacker maintains access
 Credential Access	Passwords dumped	Credential Dumping	T1003	Steal credentials from memory
 Lateral Movement	Access other systems	Remote Services	T1021	Move across the network
 Command & Control	Communicate externally	Web Protocols	T1071.001	Connect to attacker server
 Exfiltration	Funds transferred out	Exfiltration Over C2 Channel	T1041	Data or funds leave the network

Each attack phase maps to a MITRE ATT&CK technique. The table connects attacker behaviour to detection opportunities and log sources.

Phase	Behaviour	Technique	ID	Detection Opportunity
Initial Access	Phishing email w/ malicious PDF	Phishing Attachment	T1566.001	Flag suspicious senders / attachments
Execution	User opens malicious file	User Execution	T1204	Office app spawning child processes
Execution	PowerShell malware runs	PowerShell	T1059.001	Encoded or hidden PowerShell commands
Persistence	Scheduled task created	Scheduled Task	T1053.005	Event ID 4698 – new scheduled tasks
Credential Access	Passwords dumped from memory	Credential Dumping	T1003	LSASS access via Sysmon Event ID 10
Lateral Movement	Remote access to systems	Remote Services	T1021	Logins from unusual IPs / locations
C2	Attacker communicates externally	Web Protocols	T1071.001	Repeated beaconing to external IPs
Exfiltration	Crypto funds transferred out	Exfiltration Over C2	T1041	Large outbound data transfer volumes

4. DETECTION ENGINEERING (Splunk)

4. DETECTION ENGINEERING (SIMPLIFIED)	WHAT TO DETECT	LOG SOURCES	DETECTION BENEFITS
Defenders can detect suspicious behavior at multiple stages.	 Phishing Attempts Suspicious emails and attachments  Malware Execution Unexpected programs or scripts running  Credential Theft Access to sensitive processes  Lateral Movement Logins from unusual locations  Data Exfiltration Large or unusual outbound activity	 Email Gateway Logs  Windows Event Logs  Sysmon Logs  Authentication Logs  Firewall / Proxy Logs	 Early detection of attacks  Reduce dwell time  Prevent data / fund loss  Improve security visibility  Strengthen incident response

Splunk queries that translate attacker behaviour into actionable detections, one per attack stage.

■ Phishing – Suspicious Attachment Execution

```
index=windows EventCode=4688
| search parent_process_name="winword.exe" OR parent_process_name="outlook.exe"
| stats count by user, process_name, parent_process_name
```

Word/Outlook launching other processes — likely phishing payload.

■ Execution – Encoded PowerShell

```
index=windows EventCode=4688
| search process_name="powershell.exe"
| search command_line="*encoded*" OR command_line="*-enc*"
| stats count by user, host
```

Hidden/encoded PowerShell commands — common Lazarus malware delivery.

■ Credential Access – LSASS Dump

```
index=sysmon EventCode=10
| search TargetImage="lsass.exe"
| stats count by SourceImage
```

Access to LSASS memory — how Windows passwords are stolen.

■ Persistence – Scheduled Task

```
index=windows EventCode=4698
| stats count by user, task_name, host
```

New scheduled tasks — used to maintain persistent access.

■ Lateral Movement – Unusual Logins

```
index=windows EventCode=4624
| stats count by user, src_ip, host
| where count > 10
```

Repeated logins from same user/IP — possible movement across systems.

■ Command & Control – Beaconing

```
index=network
| stats count by src_ip, dest_ip
| where count > 100
```

Repeated outbound connections — attacker C2 communication.

■ Exfiltration – Large Data Transfer

```
index=network
| stats sum(bytes_out) by src_ip
| where sum(bytes_out) > 100000000
```

Unusually large outbound traffic — crypto/data exfiltration.

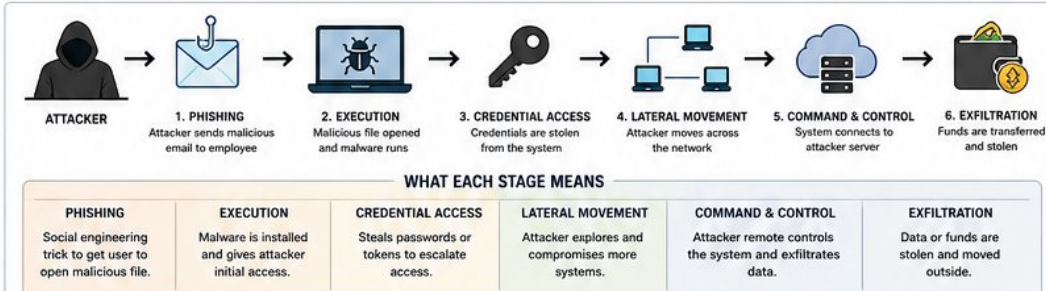
5. ATTACK CHAIN DIAGRAM

5. ATTACK DIAGRAM (EXPLANATION)

This diagram shows the full attack chain used by Lazarus Group in the Ronin Hack.

Flow:

1. Entry through phishing
2. Malware execution
3. Credential theft
4. Lateral movement
5. Command & Control
6. Exfiltration of funds



The diagram above shows the full end-to-end attack chain used by Lazarus Group in the Ronin Hack. Each stage feeds the next: phishing gains a foothold, malware executes, credentials are stolen, the attacker moves laterally across the network, establishes command and control, and finally exfiltrates funds.

6. KEY FINDINGS

✓ Human Vulnerability Was the Entry Point

The attack started with social engineering — a fake job offer, not a technical exploit. One employee opening a file gave attackers full network access.

✓ Legitimate Tools Were Abused (LOLBins)

Lazarus relied on built-in Windows tools — PowerShell and scheduled tasks — making detection harder as the activity blends with normal operations.

✓ Credential Theft Was the Turning Point

Once credentials were stolen, attackers moved freely across the network. Detection should prioritise this stage above all others.

✓ Lack of Monitoring Enabled Success

No alerts were triggered on validator access or unusual transactions. Visibility at this layer would have stopped the theft.

✓ Detection Was Possible at Every Stage

Phishing, execution, credential dumping, and lateral movement all offered detection opportunities — none were in place.